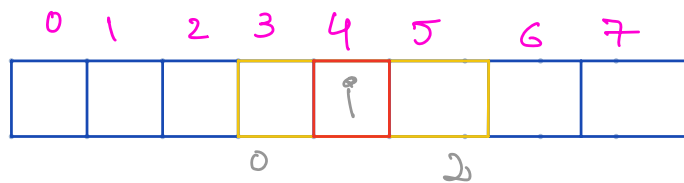


Background

$$y_i = \sum_{j=0}^{2r} f_j * x_{i+j-r}$$



$i-j$

$$2(1)+1=3$$

$$4+0-1=3$$

$$4+1-1=4$$

$$4+2-1=5$$

$$P_{yx} = \sum_{i=0}^{2r_y} \sum_{j=0}^{2r_x} f_{i,j} * N_{y-r_y+i, x-r_x+j}$$

Parallel convolution: a basic algorithm

```

01 __global__ void convolution_2D_basic_kernel(float *N, float *F, float *P,
02     int r, int width, int height) {
03     int outCol = blockIdx.x*blockDim.x + threadIdx.x;
04     int outRow = blockIdx.y*blockDim.y + threadIdx.y;
05     float Pvalue = 0.0f;
06     for (int fRow = 0; fRow < 2*r+1; fRow++) {
07         for (int fCol = 0; fCol < 2*r+1; fCol++) {
08             inRow = outRow - r + fRow;
09             inCol = outCol - r + fCol;
10             if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
11                 Pvalue += F[fRow*(2*r+1)+fCol] * N[inRow*width + inCol];
12             }
13         }
14     }
15     P[outRow][outCol] = Pvalue;
16 }

```

Diagram illustrating the memory access pattern for the convolution kernel. The expression $F[fRow*(2*r+1)+fCol]$ is annotated with "4 bytes" and "FLOP". The expression $N[inRow*width + inCol]$ is also annotated with "4 bytes" and "FLOP". A red bracket connects the two expressions, indicating the multiplication operation.

$$\text{Flops/B} = \frac{4+4}{1+1}$$

$$= 0.25$$

Tiled convolution with halo cells

